

Cell sensitive to position in line:

The sole importance of the crossing of the Berezina lies in the fact that it plainly and indubitably proved the fallacy of all the plans for cutting off the enemy's retreat and the soundness of the only possible line of action--the one Kutuzov and the general mass of the army demanded--namely, simply to follow the enemy up. The French crowd fled at a continually increasing speed and all its energy was directed to reaching its goal. It fled in such a panic and it was impossible to block its path. This was the result of the arrangements it made for crossing as by what took place at the bridges. When the bridges broke down, unarmed men, women with children who were with the French transport, all--carried on by vis inertiae--pressed forward into boats and into the ice-covered water and did not, surrender.

S&DS 365 / 665

Intermediate Machine Learning

Cell that turns on inside quote

"You mean to imply, I mean to say, that I am a man of no honor. On the contrary, I can supply you with everything even if you want to give dinner parties," warmly replied Chinatou, who tried by every word he spoke to prove his own rectitude. He then imagined Kutuzov to be animated by the same desire.

RNNs, LSTMs, GRUs, and All That

Kutuzov, shrugging his shoulders, replied with his subtle penetrating smile: "I meant merely to say what I said."

Cell that robustly activates inside if statements:

November 12

```
static int __dequeue_signal(struct sigpending *pending, sigset_t *mask,
                           siginfo_t *info)
{
    int sig = next_signal(pending, mask);
    if (sig) {
        if (current->notifier) {
            if (sigismember(current->notifier_mask, sig)) {
                if (!current->notifier)(current->notifier_data)) {
                    clear_thread_flag(TIF_SIGPENDING);
                    return 0;
                }
            }
        }
        collect_signal(sig, pending, info);
    }
    return sig;
}
```

Yale

Home stretch!

- Assignment 4 due next week
- Assignment 5 (last!) posted today
- Quiz 5 (last!) next Wednesday, November 19

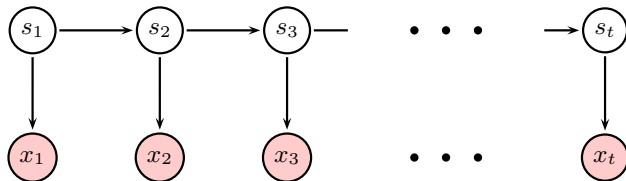
Recall: Hidden Markov Model

In an HMM, the next word is generated from a latent variable

- State is chosen stochastically (that is, probabilistically)
- Introduces dependence on earlier words that are further back than the previous time (non-Markov on observations)

Hidden Markov Model

The graphical model looks like this:



Here x_t is observable (word) at time t and s_t is unobserved state.

Hidden Markov Models

Probability of word sequence:

$$p(x_1, \dots, x_n) = \sum_{s_1, \dots, s_n} \prod_{t=1}^n p(s_t | s_{t-1}) p(x_t | s_t)$$

Hidden Markov Models

Probability of word sequence:

$$p(x_1, \dots, x_n) = \sum_{s_1, \dots, s_n} \prod_{t=1}^n p(s_t | s_{t-1}) p(x_t | s_t)$$

Probability of the next word:

$$p(x_t | x_1, \dots, x_{t-1}) = \sum_{s_t} p(s_t | x_1, \dots, x_{t-1}) p(x_t | s_t)$$

This is efficiently computed using dynamic programming

Hidden Markov Models

Probability of the next word:

$$p(x_t | x_1, \dots, x_{t-1}) = \sum_{s_t} p(s_t | x_1, \dots, x_{t-1}) p(x_t | s_t)$$

This is efficiently computed using dynamic programming:

$$p(s_t | x_1, \dots, x_{t-1}) \propto \sum_{s_{t-1}} p(s_{t-1} | x_1, \dots, x_{t-2}) p(x_{t-1} | s_{t-1}) p(s_t | s_{t-1})$$

Hidden Markov Model

- Next word x_t is predicted from the state s_t
- State is a summary of information from the past $x_1, \dots, x_{t-1}$
- State evolves stochastically

Hidden Markov Models: Estimation

The usual algorithms for estimating the parameters are iterative:

- Use the forward-backward algorithm to compute the probability that the outputs are generated from each state
- Treat the data as labeled, with these weights, and compute the relative frequencies
- Repeat
- Maximum likelihood (or MAP) estimation

Recurrent neural networks

- Can be thought of as a type of language model
- Similar to hidden Markov models, but the *state evolves deterministically*, not stochastically
- HMMs are mixture models; RNNs are not
- The state is distributed, not categorical as for HMMs
- We'll describe this using characters rather than words — could do either

HMMs vs RNNs

In an HMM:

$$p(x_t | x_1, \dots, x_{t-1}) = \sum_{s_t} p(s_t | x_1, \dots, x_{t-1}) p(x_t | s_t)$$

This is efficiently computed using dynamic programming

HMMs vs RNNs

In an HMM:

$$p(x_t | x_1, \dots, x_{t-1}) = \sum_{s_t} p(s_t | x_1, \dots, x_{t-1}) p(x_t | s_t)$$

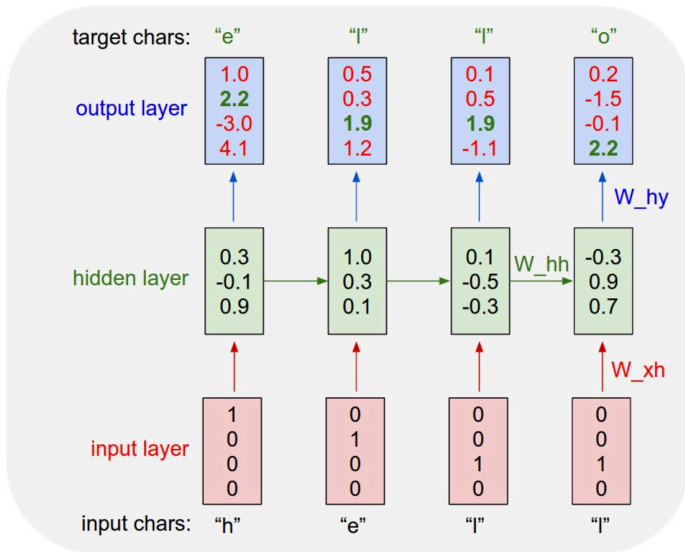
This is efficiently computed using dynamic programming

In an RNN:

$$p(x_t | x_1, \dots, x_{t-1}) = p(x_t | s_t)$$

where $s_t = s_\theta(x_1, \dots, x_{t-1})$ is a (deterministic) nonlinear function

RNNs



RNNs

This means

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

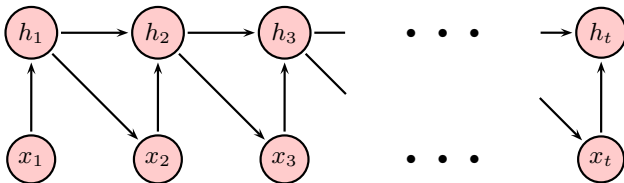
$$y_t = W_{hy}h_t + b_y$$

$$x_{t+1} | y_t \sim \text{Multinomial}(\pi(y_t))$$

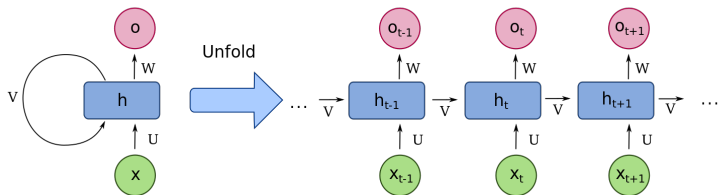
where $\pi(\cdot)$ is the soft-max function.

In this illustration, x_t is the “1-hot” representation of a character, $W_{xh} \in \mathbb{R}^{3 \times 4}$, $W_{hh} \in \mathbb{R}^{3 \times 3}$ and $W_{hy} \in \mathbb{R}^{4 \times 3}$.

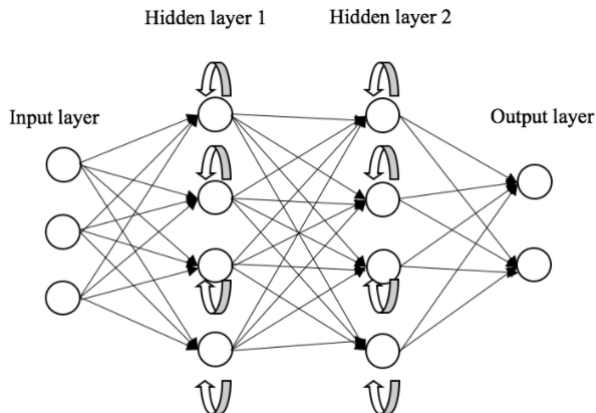
RNN graphical structure: Different views



RNN graphical structure: Different views



RNN graphical structure: Different views



General Form of RNNs

Loss function

The model is trained to assign high probability to the word (or character) that appears next:

$$\text{Loss} = -\frac{1}{T} \sum_{t=1}^T \log p(x_t | h_t)$$

If the last layer assigns a score to word v as

$$f(v, h_t) = \beta_v^T h_t$$

then this is given by

$$\text{Loss} = -\frac{1}{T} \sum_{t=1}^T \left\{ \beta_{x_t}^T h_t - \log \left(\sum_{v \in V} e^{\beta_v^T h_t} \right) \right\}$$

In terms of our earlier notation, β_v^T would be the v -th row of the matrix W_{hy} , and the intercept would be zero

Interpreting RNNs

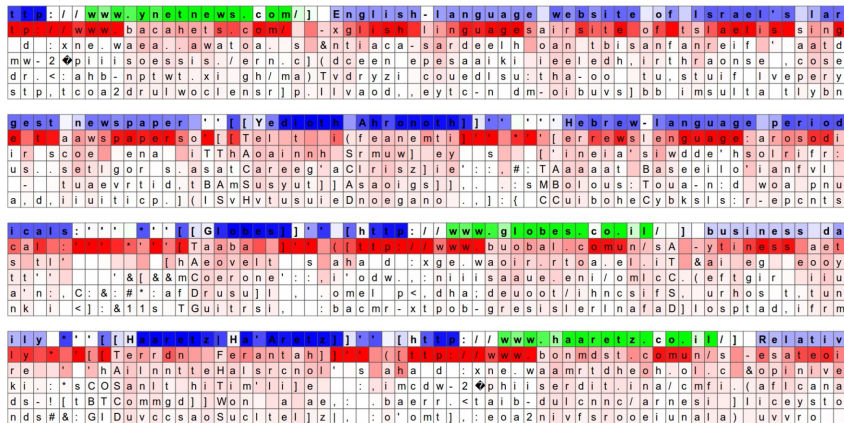
Each component is a tanh-thresholded linear function of the previous states and current input—a “neuron”

If the j th neuron h_{tj} is close to ± 1 then it is very “excited.” This is colored green in the following illustration. If it is near zero, then it is “not excited” or cool—this is colored blue.

Below the neuron color, the five most probable characters c are shown — ordered in decreasing order of y_{tc} and hence $\pi(y_t)_c$.

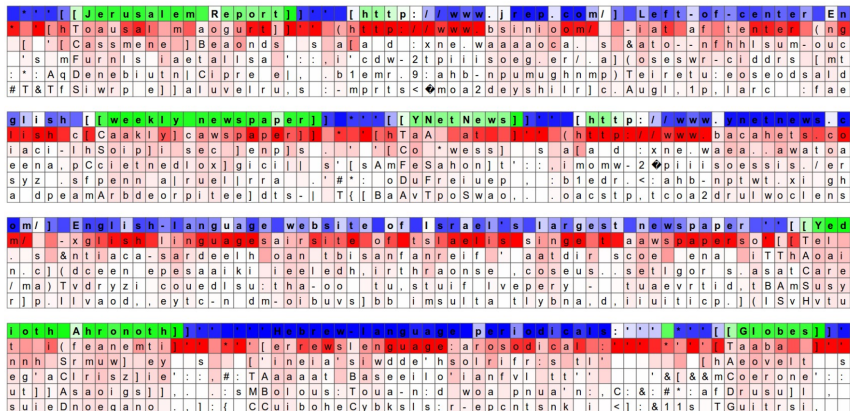
Three layers, with 512 neurons on each level. Most will be completely uninterpretable. Recall—it’s generally even hard to interpret coefficients in a least squares linear regression model!

Interpretation



The neuron highlighted in this image seems to get very excited about URLs and turns off outside of the URLs. The LSTM is likely using this neuron to remember if it is inside a URL or not.

Interpretation



The highlighted neuron here gets very excited when the RNN is inside the [[]] markdown environment and turns off outside of it.

Interestingly, the neuron can't turn on right after it sees the character "[", it must wait for the second "[" and then activate. This task of counting whether the model has seen one or two "[" is likely done with a different neuron.

Using RNNs to model mathematical equations

- We combined topic modeling for text and recurrent neural networks for equations
- The topics of a scientific article were inferred from the words
- Conditioned on the topics, the mathematical equations were generated using an RNN

Using RNNs to model mathematical equations

Proceedings of the AAAI Conference on Artificial Intelligence

Current Archives About ▾

[Home](#) / [Archives](#) / Vol. 33 No. 01: AAAI-19, IAAI-19, EAAI-20 / AAAI Technical Track: Natural Language Processing

TopicEq: A Joint Topic and Mathematical Equation Model for Scientific Texts

Michihiro Yasunaga

Yale University

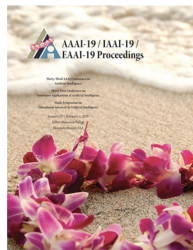
John D. Lafferty

Yale University

DOI: <https://doi.org/10.1609/aaai.v33i01.33017394>

Abstract

Scientific documents rely on both mathematics and text to communicate ideas. Inspired by the topical correspondence between mathematical equations and word contexts observed in scientific texts, we propose a novel topic model that jointly generates mathematical equations and their surrounding text (*TopicEq*). Using an extension of the correlated topic model, the context is generated from a mixture of latent topics, and the equation is generated by an RNN that depends on the latent topic



PDF

Using RNNs to model mathematical equations

Quantum physics	spin energy field electron magnetic state states hamiltonian
Particle physics	higgs neutrino coupling decay scale masses mixing quark
Astrophysics	mass gas star stellar galaxies disk halo radius luminosity
Relativity	black metric hole schwarzschild gravity holes einstein
Number theory	prime integer numbers conjecture integers degree modulo
Graph theory	graph vertex vertices edges node edge number set tree
Linear algebra	matrix matrices vector basis vectors diagonal rank linear
Optimization	problem optimization algorithm function solution gradient
Probability	random probability distribution process measure time
Machine learning	layer word image feature sentence model cnn lstm training

Using RNNs to model mathematical equations

Latent topic vector θ included as additional information in RNN state transitions:

$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t + W_{x\theta}\theta_d) \\y_t &= W_{hy}h_t \\x_{t+1} | y_t &\sim \text{Multinomial}(\pi(y_t))\end{aligned}$$

where $\pi(\cdot)$ is the soft-max function.

Topic modeling for equations

Topic	Generated Equations
Quantum physics	• $E = \hbar \frac{\partial^2 S}{\partial t^2} \left(\frac{\partial \varphi}{\partial c} \right) - \frac{k}{\hbar^2} \frac{\partial B}{\partial t} (t + \partial_t \delta).$ • $\Psi_{\text{pr}} = \sum_{\mathbf{l}} (\psi_{\mathbf{r}+\uparrow} - \psi_{\mathbf{r}\downarrow}^\dagger) + \sum_{\mathbf{r}'} (\psi_{\mathbf{r}\downarrow,\uparrow}^\dagger - \psi_{\mathbf{r}\downarrow} \sigma^\dagger).$
Particle physics	• $\mathcal{H} = \frac{2}{4}(\partial_\mu \phi)^2 + 2m\phi_\nu(\phi) + \frac{1}{2}m^2(\phi)(1 - \phi^2)^2.$ • $m_{\text{eff}}(M) = 1.4 \cdot 10^{-13} \text{ GeV}.$
Relativity	• $\mathcal{M} = \frac{2}{2}g^{\mu\nu}(f_{\mu\nu,\mu} - g_{\mu\nu,\nu} + g_{\nu\nu,b}f_{\mu,\nu}) + \frac{1}{2}g^{\mu\nu}.$ • $T_{\mu\nu} = \int_0^\infty ds_{\mu\nu} ds^2 + a_\mu^2 dr^2 + r^2 d\Omega^2.$

LSTM with two hidden layers, 512 neurons in each layer, trained on 40,000 scientific articles, using $\text{\LaTeX}$ source. The generated $\text{\LaTeX}$ compiles about 80% of the time; otherwise needs minor tweaks.

Topic modeling for equations

Number theory	• $(2^k)^k + (1^n + 1)(1 + p^k) = 1.$
Linear algebra	• $\text{tr}(E_\varepsilon X^*) = U^\top (\text{tr}(V_\varepsilon X)).$ • $\phi_h(\theta, y) = \left\{ X \in \text{Span} \left(P_c(\mathbf{T}[x, \mathbf{x}]) \right) \right\}.$
Optimization	• $\min_p p(x)$ subject to $\ p^x - y\ _2 \leq m_p.$ • $w^+ = w_t + g_t \ u_t - \nabla \mathbf{u}^*\ _2^2.$
Probability	• $\mathbb{P}(r_\tau < t) = \mathbb{E}_{\tau_{\text{wist}}} (N_\tau).$ • $T^*(t) = \lim_{t \rightarrow \infty} \mathbb{E}[N(t) + \mathbb{E}[\varphi_t(x)]^*]$

LSTM with two hidden layers, 512 neurons in each layer, trained on 40,000 scientific articles, using $\text{\LaTeX}$ source. The generated $\text{\LaTeX}$ compiles about 80% of the time; otherwise needs minor tweaks.

Topic modeling for equations

Context words	Inferred Topics	Generated Equations
star gravity einstein mass galaxies	58% Astrophysics 36% Relativity 3% Quantum physics	$\left(\frac{m_b}{M_\odot}\right)^{b_\nu} \ll g \sqrt{\frac{\tilde{\Phi}_0}{\eta_{\text{eff}}}}.$ $G(r) = \int_{r_0}^{r_0} dr \sqrt{\log(r)(\bar{r} + r_0(w))}.$
data training likelihood model gradient	62% Machine learning 21% Statistics 15% Optimization	$L = -\frac{1}{N} \sum_{i=1}^N \mathcal{R}_{R_i}(\mathbf{r}_i) + V^r(\mathbf{r}_i).$ $\operatorname{argmax}_{\mathcal{U}} \mathbb{E}_{W \sim \Psi} \log \exp [\Lambda(\widetilde{W}) - H].$

LSTM with two hidden layers, 512 neurons in each layer, trained on 40,000 scientific articles, using $\text{\LaTeX}$ source. The generated $\text{\LaTeX}$ compiles about 80% of the time; otherwise needs minor tweaks.

Fake ν_s

Which is fake?

$$L = \int_0^{e^{T-1}} \psi \sin^2 \theta d\lambda \sin^2 \theta d\theta^2 - C^{2/\sigma}$$

$$L = \int_0^L dz \|x_z\| = \int_0^L dz \sqrt{x_z \cdot x_z} \equiv \int_0^L dz \sqrt{g}$$

Fake ν_s

$$L = \int_0^{e^{T-1}} \psi \sin^2 \theta d\lambda \sin^2 \theta d\theta^2 - C^{2/\sigma}$$

$$L = \int_0^L dz \|x_z\| = \int_0^L dz \sqrt{x_z \cdot x_z} \equiv \int_0^L dz \sqrt{g}$$

Fake ν_s

Which is fake?

$$(xy^{-1})_{N(y)} = s^\epsilon(sxy^{-1})_{N(y)} = s^\epsilon(sx)_\emptyset y_\emptyset^{-1}$$

$$(xy^2 - x^2)(y(x) - Ty(x)) \leq \int_0^T e^{-x't(w)} dt$$

Fake ν_s

$$(xy^{-1})_{N(y)} = s^\epsilon(sxy^{-1})_{N(y)} = s^\epsilon(sx)_\emptyset y_\emptyset^{-1}$$

$$(xy^2 - x^2)(y(x) - Ty(x)) \leq \int_0^T e^{-x't(w)} dt$$

Fake ν_s with topics

“optimization”

$$X_L = -\frac{1}{\sum_i y_i} \pi(y_i, x_i, x > 0) \text{ subject to } x_i^T X^H x_i \leq \epsilon^2$$

$$\text{minimize } -\sum_{i=1}^K \sum_{c=1}^Z a_{mij} \|g_i\|_2^2$$

Fake ν_s with topics

“physics”

$$\hat{b}_{\text{prc}} = \frac{\hbar^2}{R} - \mu_{\text{sit}} F_d$$

$$\frac{\partial \delta C}{\partial r} Re_a a_\mu - \int_{V_{bor, quiet}} \langle K^{\alpha \beta} \rangle_\alpha^\beta = \mathcal{H} E r + \beta_{\alpha \mu \nu}^\top R_{\text{conseds}}^{\mu \lambda} \partial_\mu$$

$$\rho_{deg} \propto 11eV \left[L(003B)/20 \pm 10 if s. 1.8V(10^{-2} ergs^{-1})/5; 1.13neV \right]$$

rnn-demo.ipynb: numpy version



We'll train an "np-complete" RNN on the complete works of William Shakespeare, downloaded from [Project Gutenberg](#) (specifically, [this link](#)).

This uses a simple, direct implementation of backpropagation for RNNs, paralleling what we did for simple feedforward networks.

```

In [ ]: def lossFun(inputs, targets, hprev):
    """
    inputs, targets are both list of integers.
    hprev is Hx1 array of initial hidden state
    returns the loss, gradients on model parameters, and last hidden state
    """

    xs, hs, ys, ps = {}, {}, {}, {}
    hs[-1] = np.copy(hprev)
    loss = 0

    # forward pass
    for t in range(len(inputs)):
        xs[t] = np.zeros((vocab_size,1)) # encode in 1-of-k representation
        xs[t][inputs[t]] = 1
        hs[t] = np.tanh(np.dot(Wxh, xs[t]) + np.dot(Whh, hs[t-1]) + bh) # hidden state
        ys[t] = np.dot(Why, hs[t]) + by # unnormalized log probabilities for next chars
        ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
        loss += -np.log(ps[t][targets[t],0]) # softmax (cross-entropy loss)

    # backward pass: compute gradients going backwards
    dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
    dbh, dby = np.zeros_like(bh), np.zeros_like(by)
    dhnext = np.zeros_like(hs[0])
    for t in reversed(range(len(inputs))):
        dy = np.copy(ps[t])
        dy[targets[t]] -= 1 # backprop into y. see http://cs231n.github.io/neural-networks-case-study/
        dWhy += np.dot(dy, hs[t].T)
        dby += dy
        dh = np.dot(Why.T, dy) + dhnext # backprop into h
        dhraw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity
        dbh += dhraw
        dWxh += np.dot(dhraw, xs[t].T)
        dWhh += np.dot(dhraw, hs[t-1].T)
        dhnext = np.dot(Whh.T, dhraw)

    for dparam in [dWxh, dWhh, dWhy, dbh, dby]:
        np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients

    return loss, dWxh, dWhh, dWhy, dbh, dby, hs[len(inputs)-1]

```


Tensorflow implementation

Text generation with an RNN

 Run in Google Colab

 [View source on GitHub](#)

 Download notebook

This tutorial demonstrates how to generate text using a character-based RNN. You will work with a dataset of Shakespeare's writing from Andrej Karpathy's [The Unreasonable Effectiveness of Recurrent Neural Networks](#). Given a sequence of characters from this data ("Shakespear"), train a model to predict the next character in the sequence ("e"). Longer sequences of text can be generated by calling the model repeatedly.

★ **Note:** Enable GPU acceleration to execute this notebook faster. In Colab: *Runtime > Change runtime type > Hardware accelerator > GPU*. If running locally make sure TensorFlow version ≥ 1.11 .

This tutorial includes runnable code implemented using [tf.keras](#) and [eager execution](#). The following is sample output when the model in this tutorial trained for 30 epochs, and started with the string "Q":



```
QUEENE:
I had thought thou hadst a Roman; for the oracle,
Thus by All bids the man against the word,
Which are so weak of care, by old care done;
Your children were in your holy love,
And the precipitation through the bleeding throne.

BISHOP OF ELY:
Marry, and will, my lord, to weep in such a one were prettiest;
Yet now I was adopted heir
Of the world's lamentable day,
To watch the next way with his father with his face?

ESCALUS:
The cause why then we are all resolved more sons.
```

Remembrance of Things Past

In principle the state h_t can carry information from far in the past.

In practice, the gradients vanish (or explode) so this doesn't really happen

We need other mechanisms to “remember” information from far away and use it to predict future words

Vanilla RNNs

State is updated according to

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

We'll modify this with a “neural circuit” for storing information to be used downstream

GRUs

“Gated Recurrent Units” (GRUs) provide a memory mechanism for longer-range dependencies than vanilla RNNs have.

Another such memory circuit is called an LSTM. We'll go through this in detail for GRUs, which are simpler, more efficient, and more commonly used.

(And this is what you'll use on Assignment 5!)

Hadamard product

We'll need to use pointwise products. This is given the fancy name “Hadamard product” and written $\odot$:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \odot \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 4 \\ 10 \\ 18 \end{pmatrix}$$

Hadamard product

We'll need to use pointwise products. This is given the fancy name "Hadamard product" and written $\odot$:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} \odot \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \\ 0 \\ 4 \end{pmatrix}$$

Gated recurrent units (GRUs)



High level idea:

- Learn when to update hidden state to “remember” important pieces of information
- Keep them in memory until they are used
- Reset or “forget” this information when no longer useful

GRUs

GRUs make use of “gates” denoted by Γ (Greek G for “Gate”)

$\Gamma = 1$: “the gate is open” and information flows through

$\Gamma = 0$: “the gate is closed” and information is blocked

GRUs

Two types of gates are used:

Γ^u : When open, information from long-term memory is propagated. When closed, information from local state is used.

Γ^r : When closed, the local state is reset. When open, the state is updated as in a “vanilla” RNN.

Note: These are usually called the “update” and “reset” gates. I find this terminology super confusing. More suggestive names might be the long distance memory gate (Γ^u) and the short distance memory gate (Γ^r).

GRUs

The state evolves according to

$$c_t = \tanh (W_{xh}x_t + \Gamma_t^r \odot W_{hh}h_{t-1} + b_h)$$

$$h_t = (1 - \Gamma_t^u) \odot c_t + \Gamma_t^u \odot h_{t-1}$$

GRUs

The state evolves according to

$$c_t = \tanh (W_{xh}x_t + \Gamma_t^r \odot W_{hh}h_{t-1} + b_h)$$

$$h_t = (1 - \Gamma_t^u) \odot c_t + \Gamma_t^u \odot h_{t-1}$$

- c_t is the “candidate state” computed using the usual “vanilla RNN” state, after possibly resetting some components.
- When the long-term memory gate is open ($\Gamma^u = 1$), the information gets sent through directly. *This deals with vanishing gradients.*
- The gates are multi-dimensional, applied componentwise
- Prediction of the next word is made using h_t .

GRUs

Everything needs to be differentiable, so the gate is actually “soft” and between zero and one.

The gates are computed as

$$\Gamma_t^u = \sigma(W_{ux}x_t + W_{uh}h_{t-1} + b_u)$$

$$\Gamma_t^r = \sigma(W_{rx}x_t + W_{rh}h_{t-1} + b_r)$$

where σ is the sigmoid function.

Putting it together

GRU state update equations

$$\Gamma_t^u = \sigma(W_{ux}x_t + W_{uh}h_{t-1} + b_u)$$

$$\Gamma_t^r = \sigma(W_{rx}x_t + W_{rh}h_{t-1} + b_r)$$

$$c_t = \tanh(W_{xh}x_t + \Gamma_t^r \odot W_{hh}h_{t-1} + b_h)$$

$$h_t = (1 - \Gamma_t^u) \odot c_t + \Gamma_t^u \odot h_{t-1}$$

Putting it together

The reset gate is sometimes moved inside the linear map:

GRU state update equations

$$\Gamma_t^u = \sigma(W_{ux}x_t + W_{uh}h_{t-1} + b_u)$$

$$\Gamma_t^r = \sigma(W_{rx}x_t + W_{rh}h_{t-1} + b_r)$$

$$c_t = \tanh(W_{xh}x_t + W_{hh}(\Gamma_t^r \odot h_{t-1}) + b_h)$$

$$h_t = (1 - \Gamma_t^u) \odot c_t + \Gamma_t^u \odot h_{t-1}$$

GRUs: Example

Example:

The leaves, as the weather turned cold in New Haven, fall silently from the trees in November.

We want to keep `leaves` in memory. It's the subject of the sentence, and plural — syntax

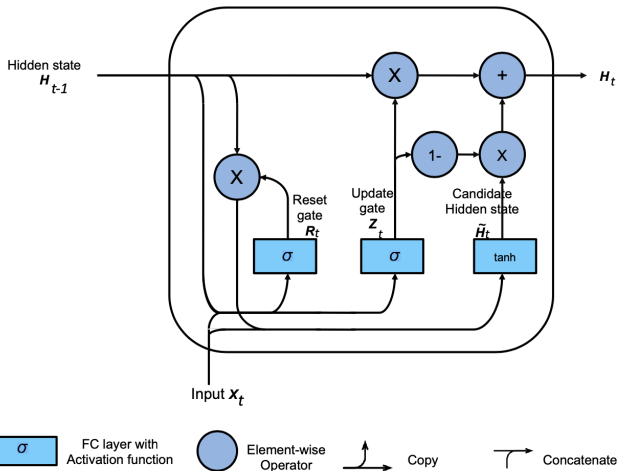
It also has a “foliage” meaning that is relevant when we predict the words `trees` and `November` — semantics

Let's step through how the GRU handles this.

GRUs: Example

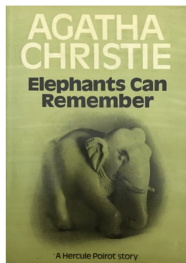
The leaves, as the weather turned cold in New Haven, fall silently from the trees in November.

GRU Diagram (using slightly different notation)



Problems on Assn5

Problem 1: Elephants Can Remember (25 points)



In this problem, we will work with "vanilla" Recurrent Neural Networks (RNNs) and Recurrent Neural Networks with Gated Recurrent Units (GRUs). The models in this part of the assignment will be character-based models, trained on an extract of the book [Elephants Can Remember](#) by Agatha Christie. To reduce the size of our vocabulary, the text is pre-processed by converting the letters to lower case and removing numbers. The code below shows some information about our training and test set. All the necessary files for this problem are available through Canvas, under the file name "problem1_data".

Problems on Assn5

Problem 2. Be the Bard (35 points)



Transformer models are the current state of the art in many sequence modeling tasks, and the Transformer architecture underlies most Large Language Models (LLMs), including ChatGPT, Llama, Mistral, etc.

In this problem, we will implement a Transformer language model from scratch in `numpy`. You will be provided with the weights of a small Transformer language model that we trained on the works of Shakespeare. We will walk through implementing each component of the Transformer architecture and ultimately assemble this into a language model that can generate some text in the style of the Bard.

Summary

- Recurrent neural networks are used for sequential data
- GRUs model longer range dependencies through special gates that control what moves in and out of memory
- RNNs are unsupervised, generative models, able to generate realistic looking sequences when sufficiently well trained.